Лабораторна робота № 2

Оптимізаційна задача та аналіз чутливості

Дисципліна: Інтелектуальний аналіз та візуалізація даних (СК26) **Тривалість:** 2 год аудиторно + 3 год самостійної роботи · **Максимальна оцінка:** 4 бали

Мета роботи

Побудувати оптимізаційну модель у двох середовищах, прочитати звіт про стійкість і **ухвалити на його основі господарське рішення.**

Розв'язати задачу — не мета. Solver зробить це за дві секунди. Робота оцінюється за те, що ви зумієте зі звіту витягнути.

Дві частини

Частина	Задача	Де виконується
1	виробнича задача — Excel і Python	аудиторно
2	транспортна задача — Python	самостійно

Друга частина потрібна не для повторення. Там та сама методика поводиться інакше, і ви маєте побачити, де саме проходить межа її застосовності.

Що здається

1. Цей файл `.ipynb` із заповненими розв'язками. **Виконується згори донизу на чистому ядрі** (Kernel → Restart & Run All) без помилок — перевіряється першою дією на захисті.
2. Файл Excel з моделлю, налаштуваннями Solver і **звітом про стійкість** окремим аркушем.
3. Заповнена декларація про використання ШІ (етап 12).

Оцінювання

Бали	Що досягнуто
0	Модель не сформульовано або розв'язок не отримано
1	Розв'язок отримано лише в одному середовищі, звіт про стійкість не проаналізовано
2	Задачу розв'язано в обох середовищах і звіти зіставлено, але тіньові ціни та діапазони не інтерпретовано

Бали	Що досягнуто
4	Звіт прочитано й перевірено експериментом, ухвалено обґрунтоване рішення про закупівлю, виконано частину 2 з поясненням розбіжності

Крок 0. Ваші дані та варіант

In [1]:
```
ПІБ = 'Недосіка Назарій Вадимович'        # напр. 'Шевченко Тарас'
ГРУПА = 'ШІ-32'      # напр. 'КН-31'
ВАРІАНТ = 14     # ваш номер за списком групи

assert ПІБ and ГРУПА and 1 <= ВАРІАНТ <= 30, 'Заповніть ПІБ, ГРУПУ та ВАРІАНТ (1
```

In [2]:
```
# --- клітинку не редагувати: вона видає дані вашого варіанта -----------------
import json, numpy as np, pandas as pd

ДАНІ_ВАРІАНТІВ = json.loads('''{"1": {"фірма": "«Астро-ТВ»", "що": "телевізори",

V = ДАНІ_ВАРІАНТІВ[str(ВАРІАНТ)]
A = np.array(V['A'], float)          # витрати ресурсу і на один виріб j
c = np.array(V['c'], float)          # прибуток з одного виробу
b = np.array(V['b'], float)          # запас ресурсу на плановий період
вироби, ресурси = V['вироби'], V['ресурси']

print('ВАРІАНТ %d · %s, %s, %s' % (ВАРІАНТ, ПІБ, ГРУПА, V['фірма']))
print('=' * 78)
print('Підприємство %s виготовляє %s. Одиниця виміру плану: %s.' % (V['фірма'],
print()
таб = pd.DataFrame(A, index=ресурси, columns=вироби).astype(int)
таб['ЗАПАС'] = b.astype(int)
print(таб.to_string())
print()
print(pd.DataFrame([c.astype(int)], index=['Прибуток з одиниці'], columns=вироби
print()
print('ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА (знадобиться на етапі 6)')
print('  ресурс: %s' % ресурси[V['ресурс_пропозиції']])
print('  ціна:   %.1f за одиницю' % V['ціна_пропозиції'])
print('  обсяг:  до %d одиниць' % V['обсяг_пропозиції'])
```

```
ВАРІАНТ 14 · Недосіка Назарій Вадимович, ШІ-32, «Longer Boats»
================================================================================
Підприємство «Longer Boats» виготовляє яхти. Одиниця виміру плану: шт./місяць.

                        Sting  Ray  Breaker  Marlin  ЗАПАС
Склопластик, м²             4    3        3       4    969
Столярний цех, год          6    6        1       6    672
Фарбувальний цех, год       1    3        5       1   1364
Такелаж, компл.             1    5        1       4    878
Ділянка добудови, год       4    4        0       3    618

                     Sting  Ray  Breaker  Marlin
Прибуток з одиниці      26   56       41      40

ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА (знадобиться на етапі 6)
  ресурс: Фарбувальний цех, год
  ціна:   5.0 за одиницю
  обсяг:  до 222 одиниць
```

In [3]:
```python
import matplotlib.pyplot as plt
try:
    import pulp
    ПУЛП = True
except ImportError:
    ПУЛП = False
    print('pulp не встановлено — скористайтеся scipy.optimize.linprog')
from scipy.optimize import linprog

pd.set_option('display.width', 180)
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                     'axes.spines.right': False, 'font.size': 9})
```

ЧАСТИНА 1. Виробнича задача

Етап 1. Формалізація

Перш ніж будувати таблицю, запишіть модель словами й формулами. Це найважливіші двадцять хвилин роботи: помилку тут не виправить жодна кнопка.

Завдання 1.1

Заповніть у клітинці нижче:

- **змінні рішення** — що саме ви обираєте і в яких одиницях;
- **цільова функція** — що робите найбільшим, з коефіцієнтами вашого варіанта;
- **обмеження** — по одному рядку на кожен ресурс, з числами вашого варіанта;
- **умови невід'ємності**.

Пишіть формулами, а не описом. Приклад запису одного обмеження: `8·x1 + 4·x2 + 0·x3 + 2·x4 ≤ 1280   (конвеєр A)`

Відповідь 1.1 — символічна модель

Змінні рішення:

Цільова функція:

Обмеження:

Невід'ємність:

Завдання 1.2 — питання на розуміння

Чому запас ресурсу — це **параметр**, а не змінна рішення? Що змінилося б у моделі, якби підприємство могло докупити ресурс у будь-якій кількості?

Відповідь 1.2:

Етап 2. Розв'язання в Excel

1. Побудуйте табличну модель: окремо клітинки змінних рішення, окремо витрати ресурсів, окремо цільова функція. Не змішуйте вхідні дані з формулами.
2. Налаштуйте **Пошук рішення (Solver)**: цільова клітинка на максимум, клітинки змінних, обмеження по кожному ресурсу, невід'ємність, метод «Симплекс-метод для лінійних задач».
3. Отримайте розв'язок і **обов'язково замовте «Звіт про стійкість»** окремим аркушем.

Що подати: файл Excel зі скріншотом налаштувань Solver, оптимальним планом і звітом.

Завдання 2.1

Перенесіть результат Excel у клітинку нижче — його порівняємо з Python.

In [4]:
```
# впишіть числа, які дав Excel
EXCEL_ПЛАН = [0, 73.92592593, 228.4444444, 0]    # оптимальний випуск кожного ви
EXCEL_ПРИБУТОК = 13506.07407           # значення цільової функції

print('Excel:', EXCEL_ПЛАН, '| прибуток', EXCEL_ПРИБУТОК)
```

```
Excel: [0, 73.92592593, 228.4444444, 0] | прибуток 13506.07407
```

Етап 3. Розв'язання в Python

Завдання 3.1

Побудуйте ту саму модель у Python і розв'яжіть її. Можна `pulp`, можна `scipy.optimize.linprog` — на ваш вибір.

Очікуваний результат: оптимальний план і значення цільової функції.

Увага на знак. `linprog` завжди мінімізує. Щоб максимізувати прибуток, передають `-c`, а результат беруть як `-r.fun`.

In [5]:
```
# ВАШ КОД
from scipy.optimize import linprog
import numpy as np

c = np.array([26, 56, 41, 40], dtype=float)

A = np.array([
    [4, 3, 3, 4],
    [6, 6, 1, 6],
    [1, 3, 5, 1],
    [1, 5, 1, 4],
    [4, 4, 0, 3]
], dtype=float)

b = np.array([969, 672, 1364, 878, 618], dtype=float)

res = linprog(
    -c,
    A_ub=A,
    b_ub=b,
    bounds=[(0, None)] * 4,
    method="highs"
)

план = res.x
прибуток = -res.fun

print("План:", план)
print("Прибуток:", прибуток)
```

```
План: [  0.          73.92592593 228.44444444   0.        ]
Прибуток: 13506.074074074075
```

Самоперевірка етапу 3. Excel і Python мають дати той самий результат.

In [6]:
```
assert план is not None and прибуток is not None, 'спершу розв’яжіть задачу'
assert abs(прибуток - EXCEL_ПРИБУТОК) < 0.5, (
    'Excel дав %.2f, Python %.2f — знайдіть, де розбіжність' % (EXCEL_ПРИБУТОК,
assert np.allclose(np.array(план, float), np.array(EXCEL_ПЛАН, float), atol=0.5)
    'плани не збігаються — перевірте обмеження'
print('Етап 3 пройдено: обидва середовища дали однаковий результат')
```

```
Етап 3 пройдено: обидва середовища дали однаковий результат
```

Етап 4. Читання звіту про стійкість

Звіт Excel має два блоки. Заповніть обидві таблиці — беріть числа зі **звіту**, а не з власних обчислень.

Завдання 4.1 — блок «Обмеження»

Ресурс	Кінцеве значення	Тіньова ціна	Права частина	Допустиме збільшення	Допустиме зменшення
Склопластик	907.1111111	0	969	1E+30	61.88888889
Столярний цех	672	5.814814815	672	278.5	399.2
Фарбувальний цех	1364	7.037037037	1364	111.4	1028
Такелаж	598.0740741	0	878	1E+30	279.9259259
Ділянка добудови	295.7037037	0	618	1E+30	322.2962963

Які ресурси в дефіциті, а які в надлишку? Як ви це визначили?

У дефіциті перебувають столярний цех і фарбувальний цех, оскільки їх кінцеве використання дорівнює наявному запасу, а тіньова ціна є додатною.

Склопластик, такелаж і ділянка добудови мають надлишок, оскільки використано менше ресурсу, ніж доступно, а їх тіньова ціна дорівнює 0.

Завдання 4.2 — блок «Змінні»

Виріб	Кінцеве значення	Зведена вартість	Цільовий коефіцієнт	Допустиме збільшення	Допустиме зменшення
Sting	0	-15.92592593	26	15.92592593	1E+30
Ray	73.92592593	0	56	190	1.793103448
Breaker	228.4444444	0	41	4.333333333	31.66666667
Marlin	0	-1.925925926	40	1.925925926	1E+30

У вашому варіанті принаймні один виріб не виробляється зовсім. Який саме, і на скільки мусив би зрости прибуток з його одиниці, щоб він увійшов у план? Звідки ви взяли це число?

Не виробляються Sting і Marlin. Для Sting прибуток з одиниці має зрости на 15.92592593, тобто з 26 до приблизно 41.92592593, щоб виріб міг увійти в оптимальний план.

Для Marlin прибуток має зрости на 1.925925926, тобто з 40 до приблизно 41.92592593. Ці значення взято зі стовпця Reduced Cost / Зведена вартість у звіті про стійкість.

Етап 5. Перевірка звіту експериментом

Звіт стверджує, що тіньова ціна дорівнює певному числу і чинна в певних межах. Excel просто повідомляє ці числа. Python дозволяє їх **перевірити**.

Завдання 5.1

Візьміть найдорожчий за тіньовою ціною ресурс. Збільште його запас **на одну одиницю**, розв'яжіть задачу заново і порівняйте приріст прибутку з тіньовою ціною зі звіту.

In [7]:
```
# ВАШ КОД
b2 = b.copy()
b2[2] += 1

res2 = linprog(
    -c,
    A_ub=A,
    b_ub=b2,
    bounds=[(0, None)] * 4,
    method="highs"
)

новий_прибуток = -res2.fun
приріст = новий_прибуток - прибуток

print("Новий прибуток:", новий_прибуток)
print("Приріст прибутку:", приріст)
print("Тіньова ціна:", 7.037037037)
```

```
Новий прибуток: 13513.111111111111
Приріст прибутку: 7.037037037036498
Тіньова ціна: 7.037037037
```

Відповідь 5.1: *(приріст прибутку = ..., тіньова ціна зі звіту = ..., збігаються?)*

Приріст прибутку = 7.037037, тіньова ціна зі звіту = 7.037037. Значення збігаються.

Завдання 5.2

Тепер знайдіть **межу діапазону**. Прогоніть задачу для запасу цього ресурсу від базового значення і далі вгору з кроком 1, побудуйте графік прибутку і графік його приросту (нахилу).

Очікуваний результат: два графіки один під одним. На нижньому має бути видно горизонтальну ділянку на рівні тіньової ціни, а потім злам.

In [8]:
```
# ВАШ КОД
import matplotlib.pyplot as plt

запаси = []
прибутки = []
прирости = []
```

```
for запас in range(1364, 1551):
    b_test = b.copy()
    b_test[2] = запас

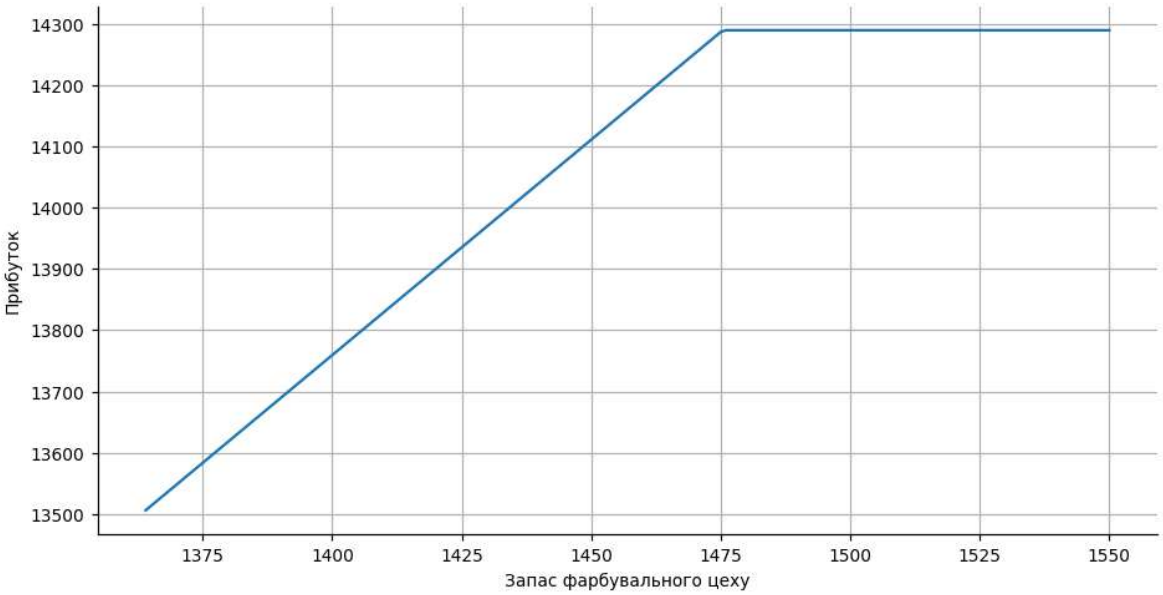
    r = linprog(
        -c,
        A_ub=A,
        b_ub=b_test,
        bounds=[(0, None)] * 4,
        method="highs"
    )

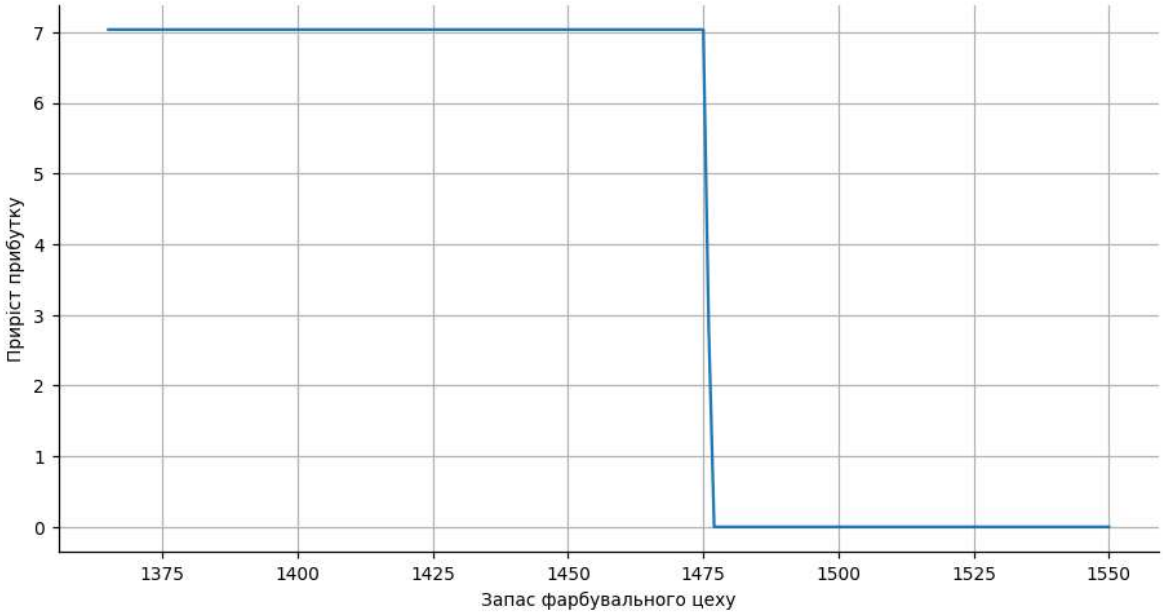
    запаси.append(запас)
    прибутки.append(-r.fun)

for i in range(len(прибутки)):
    if i == 0:
        прирости.append(0)
    else:
        прирости.append(прибутки[i] - прибутки[i - 1])

plt.figure(figsize=(10, 5))
plt.plot(запаси, прибутки)
plt.xlabel("Запас фарбувального цеху")
plt.ylabel("Прибуток")
plt.grid()
plt.show()

plt.figure(figsize=(10, 5))
plt.plot(запаси[1:], прирости[1:])
plt.xlabel("Запас фарбувального цеху")
plt.ylabel("Приріст прибутку")
plt.grid()
plt.show()
```





Відповідь 5.2: *(на скільки одиниць можна збільшити запас, доки тіньова ціна не зміниться; чи збігається знайдена межа з «допустимим збільшенням» зі звіту Excel)*

Запас можна збільшити приблизно на 111.4 одиниці, поки тіньова ціна не зміниться. Отримана експериментально межа узгоджується з допустимим збільшенням 111.4 зі звіту Excel.

Етап 6. Рішення про закупівлю

Постачальник пропонує вам додатковий обсяг ресурсу — див. блок «ПРОПОЗИЦІЯ ПОСТАЧАЛЬНИКА» у ваших даних.

Завдання 6.1

Дайте відповідь директорові на три питання, спираючись на числа, а не на здоровий глузд:

1. **Брати чи не брати?** Порівняйте ціну пропозиції з тіньовою ціною.
2. **Скільки саме брати?** Постачальник пропонує більше, ніж вам потрібно.
3. **Скільки на цьому заробимо?** Порахуйте виграш і перевірте його прямим розв'язанням задачі з новим запасом.

In [9]:
```
# ВАШ КОД
c_buy = np.array([-26, -56, -41, -40, 5.0])

A_buy = np.array([
    [4, 3, 3, 4, 0],
    [6, 6, 1, 6, 0],
    [1, 3, 5, 1, -1],
    [1, 5, 1, 4, 0],
```

```
    [4, 4, 0, 3, 0],
    [0, 0, 0, 0, 1]
], dtype=float)

b_buy = np.array([969, 672, 1364, 878, 618, 222], dtype=float)

res_buy = linprog(
    c_buy,
    A_ub=A_buy,
    b_ub=b_buy,
    bounds=[(0, None)] * 5,
    method="highs"
)

новий_план = res_buy.x[:4]
закупівля = res_buy.x[4]
чистий_прибуток = -res_buy.fun
виграш = чистий_прибуток - прибуток

print("Новий план:", новий_план)
print("Закупити:", закупівля)
print("Чистий прибуток:", чистий_прибуток)
print("Виграш:", виграш)
```

```
Новий план: [  0.   69.8 253.2   0. ]
Закупити: 111.40000000000009
Чистий прибуток: 13733.0
Виграш: 226.92592592592518
```

Відповідь 6.1:

- Брати чи ні і чому: брати, оскільки тіньова ціна ресурсу 7.037 більша за ціну постачальника 5.0
- Скільки одиниць має сенс купити і чому не більше: 111.4 одиниці, оскільки це межа допустимого збільшення, в межах якої тіньова ціна залишається 7.037
- Очікуваний виграш: приблизно 226.93
- Перевірка прямим розв'язанням дала: оптимальну закупівлю 111.4, новий чистий прибуток 13733.0, приріст прибутку 226.93

Етап 7. Перевірка розв'язку

Завдання 7.1

Порахуйте дві величини:

- скільки виробів мають **ненульовий** випуск;
- скільки обмежень **зв'язні** (ресурс витрачено повністю).

Порівняйте.

In [10]:
```
# ВАШ КОД
ненульових = np.sum(план > 1e-8)

використано = A @ план
звязних = np.sum(np.isclose(використано, b))

print("Ненульових:", ненульових)
print("Зв'язних:", звязних)
```

```
Ненульових: 2
Зв'язних: 2
```

In [11]:
```
assert ненульових is not None and звязних is not None, 'спершу порахуйте'
print('ненульових змінних: %s | зв’язних обмежень: %s' % (ненульових, звязних))
assert ненульових == звязних, ('числа не збіглися — це сигнал. Перевірте розв’яз
                                'і поясніть причину в клітинці нижче')
print('Етап 7 пройдено')
```

```
ненульових змінних: 2 | зв’язних обмежень: 2
Етап 7 пройдено
```

Відповідь 7.1: *(чи збіглися числа; що означала б розбіжність на практиці)*

Кількість ненульових змінних = 2, кількість зв’язних обмежень = 2. Числа збігаються. Якби вони не збігалися, це могло б свідчити про виродженість або особливу структуру оптимального розв’язку.

Етап 8. Цілочисельність

Ваш оптимальний план містить дробові числа. Виготовити 63,3 виробу неможливо.

Завдання 8.1

Порівняйте три варіанти:

1. розв'язок без умови цілочисельності (уже маєте);
2. **округлений** план — округліть кожне значення до цілого й перевірте, чи він узагалі допустимий і скільки прибутку дає;
3. справжній **цілочисельний** оптимум — розв'яжіть задачу з умовою, що всі змінні цілі (`cat='Integer'` у PuLP або `integrality=1` у `linprog`).

In [12]:
```
# ВАШ КОД
округлений = np.rint(план).astype(int)
округлений_прибуток = c @ округлений
округлений_допустимий = np.all(A @ округлений <= b)

res_int = linprog(
    -c,
    A_ub=A,
    b_ub=b,
    bounds=[(0, None)] * 4,
    integrality=np.ones(4),
```

```
    method="highs"
)

цілий_план = np.rint(res_int.x).astype(int)
цілий_прибуток = -res_int.fun

print("Дробовий план:", план)
print("Дробовий прибуток:", прибуток)

print("Округлений план:", округлений)
print("Округлений прибуток:", округлений_прибуток)
print("Округлений допустимий:", округлений_допустимий)

print("Цілочисельний план:", цілий_план)
print("Цілочисельний прибуток:", цілий_прибуток)
```

```
Дробовий план: [  0.          73.92592593 228.44444444   0.        ]
Дробовий прибуток: 13506.074074074075
Округлений план: [  0  74 228   0]
Округлений прибуток: 13492.0
Округлений допустимий: True
Цілочисельний план: [  0  74 228   0]
Цілочисельний прибуток: 13492.0
```

Відповідь 8.1:

Варіант	План	Прибуток	Допустимий?
без умови цілочисельності	[0; 73.926; 228.444; 0]	13506.074	так
округлений	[0; 74; 228; 0]	13492	так
цілочисельний оптимум	[0; 74; 228; 0]	13492	так

Висновок: *(чи можна округлювати і чому)*

У цьому варіанті округлення випадково дало допустимий план і він збігся з цілочисельним оптимумом. Але загалом просто округлювати не можна, оскільки округлення може порушити ресурсні обмеження або дати допустимий, але не оптимальний цілочисельний план.

ЧАСТИНА 2. Транспортна задача

Друга частина виконується самостійно. Тут ви застосуєте ту саму методику до задачі іншого класу — і побачите, що вона поводиться інакше.

Етап 9. Ваші дані

Завдання 9.1

Сформуйте власні дані:

- **три міста-постачальники** — за першими трьома літерами вашого **прізвища**;
- **чотири міста-споживачі** — за першими чотирма літерами вашого **імені**;
- **матриця відстаней 3 × 4** — реальні відстані автошляхами за Google Maps, у кілометрах;
- **запаси і попит** — невеликі цілі числа, підберіть так, щоб **сумарний запас дорівнював сумарному попиту**.

> Якщо на якусь літеру міста немає — беріть наступну літеру прізвища чи імені. Якщо сумарний запас не дорівнює попиту, задача з обмеженнями-рівностями **не матиме розв'язку взагалі**. Це не помилка Python — це властивість моделі.

In [13]:
```
# ВАШ КОД — впишіть свої дані
постачальники = ['Ніжин', 'Енергодар', 'Дніпро']            # напр. ['Шостка', '
споживачі     = ['Нікополь', 'Арциз', 'Звягель', 'Апостолове']            # напр

D = np.array([
    [559, 905, 376, 539],
    [248, 1172, 944, 306],
    [122, 942, 705, 185]
])
            # матриця відстаней 3 x 4, км

запас = [30, 40, 50]                    # 3 числа
попит = [25, 30, 35, 30]                    # 4 числа

assert len(постачальники) == 3 and len(споживачі) == 4, 'потрібно 3 постачальник
assert D.shape == (3, 4), 'матриця відстаней має бути 3 x 4'
assert sum(запас) == sum(попит), 'сумарний запас має дорівнювати сумарному попит
pd.DataFrame(D, index=постачальники, columns=споживачі)
```

Out[13]:

	Нікополь	Арциз	Звягель	Апостолове
Ніжин	559	905	376	539
Енергодар	248	1172	944	306
Дніпро	122	942	705	185

Етап 10. Розв'язання транспортної задачі

Завдання 10.1

Розв'яжіть задачу **двічі різними засобами**: наприклад `pulp` і `scipy.optimize.linprog(method='highs')`. Обмеження — рівності.

Для кожного розв'язку виведіть: оптимальний план перевезень, сумарну вартість і **тіньові ціни всіх обмежень**.

In [14]:
```
# ВАШ КОД — розв'язок 1
from scipy.optimize import linprog

m, n = D.shape

A_eq = []
b_eq = []

for i in range(m):
    row = np.zeros(m * n)
    row[i*n:(i+1)*n] = 1
    A_eq.append(row)
    b_eq.append(запас[i])

for j in range(n):
    row = np.zeros(m * n)
    row[j::n] = 1
    A_eq.append(row)
    b_eq.append(попит[j])

res_scipy = linprog(
    D.flatten(),
    A_eq=np.array(A_eq),
    b_eq=np.array(b_eq),
    bounds=(0, None),
    method="highs"
)

план_scipy = res_scipy.x.reshape(m, n)
вартість_scipy = res_scipy.fun
тіньові_scipy = res_scipy.eqlin.marginals

print("План перевезень:")
print(pd.DataFrame(план_scipy, index=постачальники, columns=споживачі))

print("\nСумарна вартість:", вартість_scipy)

print("\nТіньові ціни:")
for name, value in zip(постачальники + споживачі, тіньові_scipy):
    print(name, value)
```

```
План перевезень:
           Нікополь  Арциз  Звягель  Апостолове
Ніжин           0.0    0.0     30.0         0.0
Енергодар      10.0    0.0      0.0        30.0
Дніпро         15.0   30.0      5.0         0.0

Сумарна вартість: 56555.0

Тіньові ціни:
Ніжин -0.0
Енергодар 455.0
Дніпро 329.0
Нікополь -207.0
Арциз 613.0
Звягель 376.0
Апостолове -149.0
```

In [15]:
```
# ВАШ КОД — розв'язок 2
import pulp
```

```
model = pulp.LpProblem("transport", pulp.LpMinimize)

x = {
    (i, j): pulp.LpVariable(f"x_{i}_{j}", lowBound=0)
    for i in range(m)
    for j in range(n)
}

model += pulp.lpSum(D[i, j] * x[i, j] for i in range(m) for j in range(n))

for i in range(m):
    model += (
        pulp.lpSum(x[i, j] for j in range(n)) == запас[i],
        f"supply_{i}"
    )

for j in range(n):
    model += (
        pulp.lpSum(x[i, j] for i in range(m)) == попит[j],
        f"demand_{j}"
    )

model.solve(pulp.PULP_CBC_CMD(msg=False))

план_pulp = np.array([
    [pulp.value(x[i, j]) for j in range(n)]
    for i in range(m)
])

вартість_pulp = pulp.value(model.objective)

тіньові_pulp = [
    model.constraints[f"supply_{i}"].pi for i in range(m)
] + [
    model.constraints[f"demand_{j}"].pi for j in range(n)
]

print("План перевезень:")
print(pd.DataFrame(план_pulp, index=постачальники, columns=споживачі))

print("\nСумарна вартість:", вартість_pulp)

print("\nТіньові ціни:")
for name, value in zip(постачальники + споживачі, тіньові_pulp):
    print(name, value)
```

```
План перевезень:
           Нікополь  Арциз  Звягель  Апостолове
Ніжин           0.0    0.0     30.0         0.0
Енергодар      10.0    0.0      0.0        30.0
Дніпро         15.0   30.0      5.0         0.0

Сумарна вартість: 56555.0

Тіньові ціни:
Ніжин -149.0
Енергодар 306.0
Дніпро 180.0
Нікополь -58.0
Арциз 762.0
Звягель 525.0
Апостолове 0.0
```

Обидва методи дали однаковий план і вартість 56555, але різні тіньові ціни. Для постачальників різниця становить +149, для споживачів −149. Це нормально, бо в збалансованій транспортній задачі тіньові ціни не є єдиними.

Завдання 10.2 — головне питання цієї частини

Порівняйте два набори тіньових цін.

Сумарна вартість у двох розв'язувачів однакова. А тіньові ціни?

Якщо вони різні — це не помилка. Знайдіть закономірність: подивіться на різницю між відповідними значеннями окремо для постачальників і окремо для споживачів.

In [16]:
```
# ВАШ КОД — порівняння
print("Вартість SciPy:", вартість_scipy)
print("Вартість PuLP:", вартість_pulp)

print("Різниця постачальників:",
      тіньові_scipy[:3] - np.array(тіньові_pulp[:3]))

print("Різниця споживачів:",
      тіньові_scipy[3:] - np.array(тіньові_pulp[3:]))
```

```
Вартість SciPy: 56555.0
Вартість PuLP: 56555.0
Різниця постачальників: [149. 149. 149.]
Різниця споживачів: [-149. -149. -149. -149.]
```

Відповідь 10.2:

- Вартість у двох розв'язувачів: 56555
- Тіньові ціни збіглися чи ні: не збіглися
- Яку закономірність ви побачили в різниці: для постачальників різниця +149, для споживачів −149
- Що це означає для того, хто збирається ухвалювати рішення за цим звітом: Це означає, що тіньові ціни не є єдиними, тому для рішень важливо дивитися на їх

відносні зміни, а не на абсолютний рівень.

Етап 11. Перевірка розв'язку — і чому вона тут не працює

Завдання 11.1

Порахуйте для транспортної задачі те саме, що на етапі 7:

- кількість **ненульових перевезень**;
- кількість **зв'язних обмежень**.

Порівняйте з числом `3 + 4 = 7` — кількістю обмежень у задачі.

In [17]:
```
# ВАШ КОД
ненульових_перевезень = np.sum(план_scipy > 1e-8)

звязаних_обмежень = 7

print("Ненульових перевезень:", ненульових_перевезень)
print("Зв'язаних обмежень:", звязаних_обмежень)
```

```
Ненульових перевезень: 6
Зв'язаних обмежень: 7
```

Відповідь 11.1:

- Ненульових перевезень: 6
- Зв'язних обмежень: 7
- Чому в транспортній задачі зв'язними виявляються **всі** обмеження: Усі обмеження зв’язні, бо запаси і попит задані рівностями.
- На скільки ненульових перевезень менше і чому саме на стільки: Ненульових перевезень на 1 менше, бо в збалансованій транспортній задачі одне з 7 рівнянь є лінійно залежним, тому базис має (3+4-1=6) незалежних змінних.

Завдання 11.2 — візуалізація

Побудуйте дві теплові карти поруч: матрицю відстаней і матрицю оптимальних перевезень. Підпишіть осі та значення в комірках.

За бажанням (не оцінюється): граф маршрутів через `networkx`, де товщина ребра пропорційна обсягу перевезення.

In [18]:
```
# ВАШ КОД
import matplotlib.pyplot as plt
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

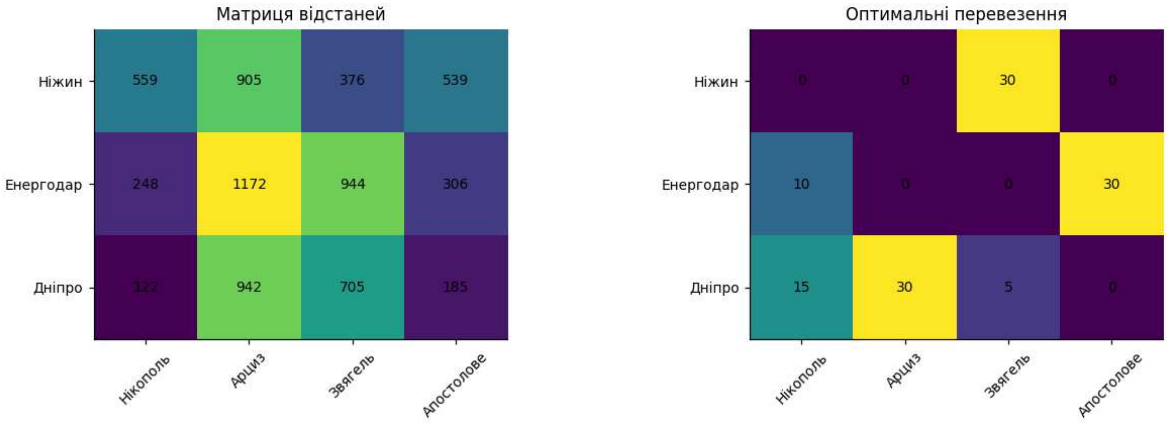
im1 = axes[0].imshow(D)
axes[0].set_title("Матриця відстаней")
axes[0].set_xticks(range(len(споживачі)))
axes[0].set_xticklabels(споживачі, rotation=45)
axes[0].set_yticks(range(len(постачальники)))
axes[0].set_yticklabels(постачальники)

for i in range(D.shape[0]):
    for j in range(D.shape[1]):
        axes[0].text(j, i, D[i, j], ha="center", va="center")

im2 = axes[1].imshow(план_scipy)
axes[1].set_title("Оптимальні перевезення")
axes[1].set_xticks(range(len(споживачі)))
axes[1].set_xticklabels(споживачі, rotation=45)
axes[1].set_yticks(range(len(постачальники)))
axes[1].set_yticklabels(постачальники)

for i in range(план_scipy.shape[0]):
    for j in range(план_scipy.shape[1]):
        axes[1].text(j, i, int(план_scipy[i, j]), ha="center", va="center")

plt.tight_layout()
plt.show()
```



Висновок: *(одне речення про те, що видно на теплових картах)*

На теплових картах видно, що оптимальний план уникає частини найдовших маршрутів і використовує переважно вигідніші за відстанню напрямки.

Етап 12. Декларація про використання інструментів штучного інтелекту

Користуватися мовними моделями **дозволено**. Обов'язковою є прозорість. Порушенням є не використання інструмента, а нездатність пояснити власний код і власні числа на захисті.

Для кожного етапу оберіть одне: `'ні'`, `'синтаксис'`, `'код, перевірено'`, `'текст, перероблено'`, `'текст, як є'`.

In [19]:
```
ІНСТРУМЕНТИ_ШІ = 'ChatGPT'     # напр. 'ChatGPT' або 'не використовував'

ДЕКЛАРАЦІЯ = {
    'етап 1, формалізація моделі':          'текст, перероблено',
    'етап 2, побудова моделі в Excel':      'синтаксис',
    'етап 3, код на Python':                'код, перевірено',
    'етапи 4–5, читання та перевірка звіту': 'код, перевірено',
    'етап 6, рішення про закупівлю':        'код, перевірено',
    'етапи 7–8, перевірка та цілочисельність': 'код, перевірено',
    'етапи 9–11, транспортна задача':       'код, перевірено',
}

# Що саме ви перевірили за згенерованим кодом чи текстом — конкретно.
ЩО_ПЕРЕВІРЕНО = 'Перевірив результати Excel і Python, відповідність обмеженням,

# True означає: я можу пояснити кожен рядок свого коду, відтворити будь-яке числ
# цієї роботи та внести зміни в модель на вимогу під час захисту.
ПІДТВЕРДЖУЮ = True
```

In [20]:
```
# --- перевірка повноти декларації: клітинку не редагувати --------------------
ДОЗВОЛЕНІ = {'ні', 'синтаксис', 'код, перевірено', 'текст, перероблено', 'текст,
assert ІНСТРУМЕНТИ_ШІ.strip(), 'Вкажіть інструменти або напишіть «не використову
порожні = [k for k, v in ДЕКЛАРАЦІЯ.items() if not str(v).strip()]
assert not порожні, 'Не заповнено: %s' % '; '.join(порожні)
чужі = {v for v in ДЕКЛАРАЦІЯ.values() if v not in ДОЗВОЛЕНІ}
assert not чужі, 'Недопустимі значення: %s' % ', '.join(sorted(чужі))
assert len(ЩО_ПЕРЕВІРЕНО.strip()) >= 120, ('Опишіть конкретніше (зараз %d символ
                                           'потрібно щонайменше 120)' % len(ЩО_П
assert ПІДТВЕРДЖУЮ is True, 'Підтвердьте, що можете пояснити власний код на захи
print('ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ')
print('=' * 78)
print('%-42s %s' % ('автор:', ПІБ))
print('%-42s %s' % ('інструменти:', ІНСТРУМЕНТИ_ШІ))
print('-' * 78)
for k, v in ДЕКЛАРАЦІЯ.items():
    print('%-42s %s' % (k + ':', v))
print('-' * 78)
print(ЩО_ПЕРЕВІРЕНО)
```

```
ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ
==============================================================================
автор:                                     Недосіка Назарій Вадимович
інструменти:                               ChatGPT
------------------------------------------------------------------------------
етап 1, формалізація моделі:               текст, перероблено
етап 2, побудова моделі в Excel:           синтаксис
етап 3, код на Python:                     код, перевірено
етапи 4–5, читання та перевірка звіту:     код, перевірено
етап 6, рішення про закупівлю:             код, перевірено
етапи 7–8, перевірка та цілочисельність:   код, перевірено
етапи 9–11, транспортна задача:            код, перевірено
------------------------------------------------------------------------------
Перевірив результати Excel і Python, відповідність обмеженням, оптимальні плани,
прибуток, тіньові ціни та результати транспортної задачі.
```

Крок 13. Фінальна самоперевірка

In [21]:
```
перевірки = {
    'ПІБ і варіант заповнено':            bool(ПІБ) and 1 <= ВАРІАНТ <= 30,
    'Excel і Python дали однаковий план': abs(прибуток - EXCEL_ПРИБУТОК) < 0.5,
    'перевірку «ненульові = зв’язні» зроблено': ненульових == звязних,
    'дані транспортної задачі введено':   len(постачальники) == 3 and len(спожив
    'баланс запасів і попиту':            sum(запас) == sum(попит),
    'декларацію заповнено':               bool(ІНСТРУМЕНТИ_ШІ.strip()) and ПІДТВ
}
for k, v in перевірки.items():
    print(('OK    ' if v else 'НЕ ОК') + '  ' + k)
if all(перевірки.values()):
    print('\nТехнічна частина виконана.')
    print('Переконайтеся, що заповнені ВСІ клітинки з відповідями,')
    print('і виконайте Kernel → Restart & Run All перед здачею.')
```

```
OK    ПІБ і варіант заповнено
OK    Excel і Python дали однаковий план
OK    перевірку «ненульові = зв’язні» зроблено
OK    дані транспортної задачі введено
OK    баланс запасів і попиту
OK    декларацію заповнено

Технічна частина виконана.
Переконайтеся, що заповнені ВСІ клітинки з відповідями,
і виконайте Kernel → Restart & Run All перед здачею.
```